EE627A Midterm

Part 1: Heuristic-Based Music Recommendation

Objective: Develop a rule-based recommendation system to predict user preferences for six candidate tracks. For each test user, you must label exactly three tracks as '1' (Recommend) and three tracks as '0' (Do Not Recommend).

1.a) Feature Engineering & Statistical Aggregation

Using the training data, calculate preference scores for each track based on the user's historical interactions with its associated **Album**, **Artist**, and **Genre(s)**.

Because a single track often belongs to multiple genres, you must implement a statistical aggregation strategy to convert the list of genre scores into a fixed set of features. You are required to generate the following feature vector for each candidate track:

- **Album Score:** The user's historical preference score for the track's album.
- **Artist Score:** The user's historical preference score for the track's artist.
- **Genre Statistics:** A set of measurements derived from the user's scores across all genres associated with the track, including but not restrict to:
    - **Count:** Number of genres associated with the track that the user has interacted with.
    - **Maximum:** The highest score among the track's genres.
    - **Minimum:** The lowest score among the track's genres.
    - **Mean:** The average score across all associated genres.
    - **Variance:** ….
    - **…**

1.b) Decision Logic & Rule Definition

Define a clear logic or "rule" to rank the six test tracks based on the features generated in step 1.a. Your rule must determine the top three tracks to be recommended.

Requirements:

- **Multiple Strategies:** Create at least two different submission files based on different heuristic rules (e.g., one focusing on *Maximum Genre Score* and another focusing on a *Weighted Average* of Album, Artist, and Mean Genre scores).
- **Justification:** Briefly document the rationale behind each rule and why you expect it to effectively capture user intent.

Example for User 249008

	TrackID		Album	Artist	Genre1	Genre2	Genre3	Genre4	Genre5	Genre6
Track 1	4967	Hierarchy List	205719	197877	172023	201738				
		History Rating	90	80						
Track 2	164591	Hierarchy List	137283	51948	33722	131552	192353	142021	224280	12274
		History Rating	80	90		80				
Track 3	127497	Hierarchy List	245158	218424	33204	239725				
		History Rating	50							
Track 4	165413	Hierarchy List	116255	276506	61215	17453	212410	201738	88853	
		History Rating	80	90		80	70			
Track 5	197975	Hierarchy List	119180	211565	33722	271945	224280			
		History Rating	80							
Track 6	239621	Hierarchy List	262661	134540	61215	212410				
		History Rating	50							

Part 2: Addressing the Cold Start Problem

Objective: Develop a strategy to provide recommendations for "cold start" users—those with sparse or non-existent training records (no historical album, artist, or genre interactions).

2.a) Strategy Design (The Global Fallback)

Explain your conceptual approach to handling users with zero historical matches. Since personalized scores are unavailable, you must design a **non-personalized (global) ranking strategy**.

- **Global Popularity:** How will you use the aggregate training data to identify "universally" liked tracks?
- **Feature Imputation:** Will you assign default scores to these users based on global averages, or will you use a different heuristic (e.g., highest average genre rating across all users)?

2.b: Strategy Design ("Dig Deeper" Search Logic)

Implement a "fallback" search logic that leverages intra-album relationships to estimate scores when direct album-level history is missing.

Explain your strategy for handling tracks where the user has no direct rating for the specific **Album ID**, but *does* have history with other tracks from that same album.

- **The Logic:** If User_Album_Score is null, your system should query the training data for all other tracks associated with that Album_ID that the user *has* interacted with.
- **Aggregation:** How will you combine those related track ratings into a single proxy score for the current candidate track? (e.g., Average rating of all known tracks in that album, or the Maximum rating?).

Implementation (Hierarchical Fallback)

Update your recommendation pipeline to follow this hierarchical priority:

1. **Primary:** Use direct Artist/Album/Genre scores.
2. **Secondary (Dig Deeper):** If Album score is missing, calculate the **Mean Track Rating** for that specific Album ID from the user's history.
3. **Tertiary (Global):** If the album is entirely new to the user, fall back to the global popularity metrics defined in Part 2.a.

Impact Analysis

Analyze whether this "Dig Deeper" strategy provides a measurable "lift" in your Kaggle score compared to a simple global fallback.

- Does looking at "sibling" tracks in an album actually predict a user's interest in a new track better than just using the Artist score?
- Conclude with an observation on the "sparsity" of your data—how often did this specific "Dig Deeper" rule actually get triggered?

Submission Format & Requirements

Objective: To provide a comprehensive technical report of your methodology, implementation, and performance results.

1. Technical Report (PDF or Word)

You must submit a single **PDF or Microsoft Word** document that serves as your project report. This document should document your entire workflow and must include:

- **Methodology:** A detailed explanation of the logic and rules used in Part 1 and Part 2.
- **Python Implementation:** The complete Python code used for feature engineering, statistical aggregation, and recommendation logic.
    - *Note: Please embed the code or provide clear snippets within the document. Do not submit standalone .py or .csv files as part of this specific requirement.*
- **Results:** A summary of the output generated by your different rule-based strategies.

2. Proof of Submission (Kaggle Scores)

To verify your participation and evaluate your model's performance, you must include evidence of your Kaggle activity:

- **Screenshots:** Capture clear screenshots of your **Kaggle Leaderboard scores** for each of your submissions.
- **Integration:** Insert these screenshots directly into your Technical Report at the end of the relevant sections (Part 1 and Part 2) to serve as submission proof.
- **Comparison:** If you made multiple submissions based on different rules, include a brief table or list comparing the scores to show which strategy performed best.